Study 3: metaperceptions, trust, and threat

Table of contents

What this document does . . . 1
Configuration . . . 1
Load data and build composites . . . 3
Benchmarks and expected credit . . . 3
Original models (mismatch score) . . . 4
New models (expected moral credit) . . . 5
Check: the two specifications are the same model . . . 6
Empathy moderation . . . 6
Figures . . . 8
Comparison with the values currently in the manuscript . . . 9
Manuscript-ready numbers . . . 9
Session info . . . 10

What this document does

The original analysis regressed trust and threat on a signed *mismatch score* (outgroup's mean rating minus the participant's metaperception for liberals; the reverse for conservatives). Within the metaperception condition the outgroup mean is a single constant, so the mismatch score is just the metaperception shifted and, for liberals, sign-flipped.

This document:

1. Rebuilds the composites exactly as in the original script.
2. Reproduces the original mismatch models (to confirm nothing changed).
3. Refits every model with the **raw metaperception** ("expected moral credit") as the predictor.
4. Verifies the two versions are statistically identical apart from sign and intercept.
5. Refits the empathy moderation models on the raw metaperception, with simple slopes at ±1 SD.
6. Compares against the older numbers that scored *proportionality* as individualizing (proportionality is binding throughout this document).
7. Prints manuscript-ready numbers.

Configuration

```
# ---- File -------------------------------------------------------------
data_path <- "/Users/yaldasmac/Desktop/Metaperception/Study\ 3/study3_data.xlsx"

# ---- Group and condition codes -----------
lib_code  <- 1   # Group == 1: liberals
con_code  <- 2   # Group == 2: conservatives
```

```r
cond_out  <- 1   # Condition == 1: outgroup perception (the benchmark)
cond_meta <- 2   # Condition == 2: metaperception

# ---- Moral foundation items ----------------------------------------
prop_in <- "binding"

core_binding <- c("loyalty_1", "loyalty_2", "authority_1", "authority_2",
                  "purity_1", "purity_2")
core_indiv   <- c("care_1", "care_2", "equality_1", "equality_2")
prop_items   <- c("proportionality_1", "proportionality_2")

# ---- Outcomes ------------------------------------------------------
trust_items  <- paste0("trust_", 1:6)
threat_items <- paste0("threat_", 1:6)
ideology_col <- "ideology"

# ---- Empathy -------------------------------------------------------
empathy_pattern <- "^em_"

# ---- Missing item responses ----------------------------------------
composite_na_rm <- FALSE
```

```r
library(readxl)
library(broom)
library(effectsize)
library(knitr)
```

```r
fmt_p <- function(p) {
  ifelse(p < .001, "< .001", sub("^0", "", sprintf("%.3f", p)))
}

check_cols <- function(df, cols, label) {
  miss <- setdiff(cols, names(df))
  if (length(miss) > 0) {
    stop(label, ": these columns are not in the data: ",
         paste(miss, collapse = ", "), call. = FALSE)
  }
}

item_sets <- function(prop_in) {
  if (prop_in == "individualizing") {
    list(binding = core_binding, indiv = c(core_indiv, prop_items))
  } else if (prop_in == "binding") {
    list(binding = c(core_binding, prop_items), indiv = core_indiv)
  } else stop("prop_in must be 'individualizing' or 'binding'")
}

# Tidy one coefficient from a model, with CI, R2, partial eta2 and a
# standardized beta.
coef_row <- function(model, term, label) {
  s   <- summary(model)
  tt  <- tidy(model, conf.int = TRUE)
  r   <- tt[tt$term == term, ]
  mf  <- model.frame(model)
  y   <- mf[[1]]
  eta <- eta_squared(model, partial = TRUE)
  eta <- eta$Eta2_partial[eta$Parameter == term]
  data.frame(
    model     = label,
    n         = nobs(model),
    df        = s$df[2],
    b         = r$estimate,
    SE        = r$std.error,
    CI_low    = r$conf.low,
    CI_high   = r$conf.high,
    t         = r$statistic,
    p         = r$p.value,
```

```
    beta_std  = r$estimate * sd(mf[[term]]) / sd(y),
    R2        = s$r.squared,
    adj_R2    = s$adj.r.squared,
    eta2_p    = eta,
    row.names = NULL
  )
}

# rbind a list of one-row data frames without carrying list names along
stack_rows <- function(lst) {
  out <- do.call(rbind, unname(lst))
  rownames(out) <- NULL
  out
}
```

Load data and build composites

```
data <- read_excel(data_path)

sets <- item_sets(prop_in)
check_cols(data, c(sets$binding, sets$indiv, trust_items, threat_items,
                   ideology_col, "Group", "Condition"), "Core variables")

data$binding         <- rowMeans(data[, sets$binding], na.rm = composite_na_rm)
data$individualizing <- rowMeans(data[, sets$indiv],   na.rm = composite_na_rm)
data$trust           <- rowMeans(data[, trust_items],  na.rm = composite_na_rm)
data$threat          <- rowMeans(data[, threat_items], na.rm = composite_na_rm)
data$ideology        <- data[[ideology_col]]

# Empathy composite
empathy_items <- grep(empathy_pattern, names(data), value = TRUE)
has_empathy   <- length(empathy_items) > 0
if (has_empathy) {
  data$empathy <- rowMeans(data[, empathy_items], na.rm = TRUE)
  cat("Empathy composite built from", length(empathy_items), "items:",
      paste(empathy_items, collapse = ", "), "\n")
} else {
  message("No columns match '", empathy_pattern,
          "': the moderation section is skipped.")
}
```

```
Empathy composite built from 14 items: em_1, em_2, em_3, em_4, em_5, em_6, em_7, em_8, em_9, em_10, em_11,
↪  em_12, em_13, em_14
```

```
lib_meta <- subset(data, Group == lib_code & Condition == cond_meta)
con_meta <- subset(data, Group == con_code & Condition == cond_meta)
con_out  <- subset(data, Group == con_code & Condition == cond_out)  # cons rating liberals
lib_out  <- subset(data, Group == lib_code & Condition == cond_out)  # libs rating conservatives
```

Benchmarks and expected credit

The predictor for each group is its metaperception on the foundation family where that group erred:

- **Liberals:** how strongly they expect conservatives to credit them with *binding* values. The benchmark is conservatives' actual binding rating of the liberal target.
- **Conservatives:** how strongly they expect liberals to credit them with *individualizing* values. The benchmark is liberals' actual individualizing rating of the conservative target.

```r
R_lib <- mean(con_out$binding,         na.rm = TRUE)  # reality for liberals
R_con <- mean(lib_out$individualizing, na.rm = TRUE)  # reality for conservatives

lib_meta$credit <- lib_meta$binding
con_meta$credit <- con_meta$individualizing

# Original mismatch scores, rebuilt exactly as before
lib_meta$mismatch <- R_lib - lib_meta$credit   # + = liberals underestimate
con_meta$mismatch <- con_meta$credit - R_con   # + = conservatives overestimate

data.frame(
  group             = c("Liberals (binding)", "Conservatives (individualizing)"),
  outgroup_actual   = c(R_lib, R_con),
  mean_expected     = c(mean(lib_meta$credit, na.rm = TRUE),
                        mean(con_meta$credit, na.rm = TRUE)),
  sd_expected       = c(sd(lib_meta$credit, na.rm = TRUE),
                        sd(con_meta$credit, na.rm = TRUE)),
  pct_below_actual  = c(100 * mean(lib_meta$credit < R_lib, na.rm = TRUE),
                        100 * mean(con_meta$credit < R_con, na.rm = TRUE)),
  n                 = c(nrow(lib_meta), nrow(con_meta))
)
```

group	outgroup_actual	mean_expected	sd_expected	pct_below_actual	n
Liberals (binding)	2.739362	2.003947	0.6735042	78.94737	95
Conservatives (individualizing)	2.212500	3.058139	0.7540815	15.11628	86

These are raw means, not the ideology-adjusted marginal means from the MANCOVA. The preregistered accuracy contrasts in Table 1 of the manuscript are unaffected by anything in this document.

Original models (mismatch score)

These should reproduce the output of the moderation script (proportionality scored as binding): liberals trust $b = -1.05$ ($R^2 = .29$), threat $b = 0.95$ ($R^2 = .23$); conservatives trust $b = 1.25$ ($R^2 = .41$), threat $b = -0.78$ ($R^2 = .15$).

```r
m_old <- list(
  lib_trust  = lm(trust  ~ mismatch + ideology, data = lib_meta),
  lib_threat = lm(threat ~ mismatch + ideology, data = lib_meta),
  con_trust  = lm(trust  ~ mismatch + ideology, data = con_meta),
  con_threat = lm(threat ~ mismatch + ideology, data = con_meta)
)

old_tab <- stack_rows(Map(coef_row, m_old, "mismatch",
  c("Liberals: trust", "Liberals: threat",
    "Conservatives: trust", "Conservatives: threat")))
old_tab$p_fmt <- fmt_p(old_tab$p)
kable(old_tab[, c("model", "n", "b", "SE", "t", "p_fmt", "R2")],
      digits = 3, caption = "Original specification (mismatch score)")
```

Table 1: Original specification (mismatch score)

model	n	b	SE	t	p_fmt	R2
Liberals: trust	95	-1.052	0.225	-4.686	< .001	0.288
Liberals: threat	95	0.947	0.253	3.749	< .001	0.225
Conservatives: trust	86	1.252	0.171	7.318	< .001	0.414
Conservatives: threat	86	-0.776	0.208	-3.733	< .001	0.150

New models (expected moral credit)

```
m_new <- list(
  lib_trust  = lm(trust  ~ credit + ideology, data = lib_meta),
  lib_threat = lm(threat ~ credit + ideology, data = lib_meta),
  con_trust  = lm(trust  ~ credit + ideology, data = con_meta),
  con_threat = lm(threat ~ credit + ideology, data = con_meta)
)

new_tab <- stack_rows(Map(coef_row, m_new, "credit",
  c("Liberals: trust", "Liberals: threat",
    "Conservatives: trust", "Conservatives: threat")))
new_tab$p_fmt <- fmt_p(new_tab$p)
kable(new_tab[, c("model", "n", "df", "b", "SE", "CI_low", "CI_high",
                  "t", "p_fmt", "beta_std", "R2", "adj_R2", "eta2_p")],
      digits = 3,
      caption = "Revised specification (expected moral credit), controlling for ideology")
```

Table 2: Revised specification (expected moral credit), controlling for ideology

model	n	df	b	SE	CI_low	CI_high	t	p_fmt	beta_std	R2	adj_R2	eta2_p
Liberals: trust	95	92	1.052	0.225	0.606	1.498	4.686	< .001	0.447	0.288	0.272	0.270
Liberals: threat	95	92	-0.947	0.253	-1.448	-0.445	-3.749	< .001	-0.373	0.225	0.209	0.203
Conservatives: trust	86	83	1.252	0.171	0.912	1.592	7.318	< .001	0.619	0.414	0.399	0.406
Conservatives: threat	86	83	-0.776	0.208	-1.189	-0.362	-3.733	< .001	-0.380	0.150	0.129	0.149

Full coefficient tables, including intercepts and ideology, for the revised SI table:

```
for (nm in names(m_new)) {
  cat("\n\n**", nm, "**\n\n", sep = "")
  print(kable(tidy(m_new[[nm]], conf.int = TRUE), digits = 3))
}
```

lib_trust

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.764	0.443	1.724	0.088	-0.116	1.643
credit	1.052	0.225	4.686	0.000	0.606	1.498
ideology	0.200	0.112	1.782	0.078	-0.023	0.423

lib_threat

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	7.647	0.498	15.351	0.000	6.657	8.636
credit	-0.947	0.253	-3.749	0.000	-1.448	-0.445
ideology	-0.233	0.126	-1.843	0.069	-0.483	0.018

con_trust

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.658	0.745	0.884	0.38	-0.823	2.140
credit	1.252	0.171	7.318	0.00	0.912	1.592
ideology	-0.109	0.081	-1.353	0.18	-0.270	0.051

con_threat

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	7.248	0.905	8.008	0.000	5.447	9.048
credit	-0.776	0.208	-3.733	0.000	-1.189	-0.362
ideology	0.038	0.098	0.389	0.698	-0.157	0.233

Check: the two specifications are the same model

Fit, residuals, and |t| should match exactly. The slope should match in size, with the sign flipped for liberals only.

```
eq <- data.frame(
  model          = names(m_new),
  R2_mismatch    = sapply(m_old, function(m) summary(m)$r.squared),
  R2_credit      = sapply(m_new, function(m) summary(m)$r.squared),
  b_mismatch     = sapply(m_old, function(m) coef(m)[["mismatch"]]),
  b_credit       = sapply(m_new, function(m) coef(m)[["credit"]]),
  t_mismatch     = old_tab$t,
  t_credit       = new_tab$t,
  same_fitted    = mapply(function(a, b) isTRUE(all.equal(fitted(a), fitted(b))),
                          m_old, m_new),
  row.names = NULL
)
kable(eq, digits = 4)
```

model	R2_mismatch	R2_credit	b_mismatch	b_credit	t_mismatch	t_credit	same_fitted
lib_trust	0.2875	0.2875	-1.0521	1.0521	-4.6864	4.6864	TRUE
lib_threat	0.2255	0.2255	0.9467	-0.9467	3.7493	-3.7493	TRUE
con_trust	0.4136	0.4136	1.2519	1.2519	7.3184	7.3184	TRUE
con_threat	0.1498	0.1498	-0.7758	-0.7758	-3.7328	-3.7328	TRUE

```
stopifnot(all(eq$same_fitted))
```

Empathy moderation

Both predictors are mean-centered within group, so each main effect is the effect at the average of the other variable. Simple slopes of expected credit are shown at −1 SD, the mean, and +1 SD of empathy.

```
moderation <- function(df, outcome, group_label) {
  df <- df[complete.cases(df[, c(outcome, "credit", "empathy", "ideology")]), ]
  df$credit_c  <- df$credit  - mean(df$credit)
  df$empathy_c <- df$empathy - mean(df$empathy)
  sd_e <- sd(df$empathy)

  f_add <- as.formula(paste(outcome, "~ credit_c + empathy_c + ideology"))
  f_int <- as.formula(paste(outcome, "~ credit_c * empathy_c + ideology"))
  m_add <- lm(f_add, data = df)
  m_int <- lm(f_int, data = df)
  cmp   <- anova(m_add, m_int)

  int <- tidy(m_int)[tidy(m_int)$term == "credit_c:empathy_c", ]

  # Simple slopes by re-centering empathy at -1 SD, mean, +1 SD
  slopes <- do.call(rbind, lapply(c(-1, 0, 1), function(k) {
    d <- df
    d$empathy_c <- d$empathy_c - k * sd_e
```

```
    r <- tidy(lm(f_int, data = d))
    r <- r[r$term == "credit_c", ]
    data.frame(empathy_level = c("-1 SD", "Mean", "+1 SD")[k + 2],
               b = r$estimate, SE = r$std.error,
               t = r$statistic, p = fmt_p(r$p.value))
  }))

  list(
    summary = data.frame(
      group       = group_label,
      outcome     = outcome,
      n           = nobs(m_int),
      b_int       = int$estimate,
      SE_int      = int$std.error,
      t_int       = int$statistic,
      df          = m_int$df.residual,
      p_int       = fmt_p(int$p.value),
      delta_F     = cmp$F[2],
      p_delta_F   = fmt_p(cmp$`Pr(>F)`[2]),
      R2_full     = summary(m_int)$r.squared
    ),
    slopes = cbind(group = group_label, outcome = outcome, slopes),
    model  = m_int
  )
}

mods <- list(
  moderation(lib_meta, "trust",  "Liberals"),
  moderation(lib_meta, "threat", "Liberals"),
  moderation(con_meta, "trust",  "Conservatives"),
  moderation(con_meta, "threat", "Conservatives")
)

kable(stack_rows(lapply(mods, `[[`, "summary")), digits = 3,
      caption = "Expected credit x empathy interactions")
```

Table 3: Expected credit x empathy interactions

group	outcome	n	b_int	SE_int	t_int	df	p_int	delta_F	p_delta_F	R2_full
Liberals	trust	95	-1.861	0.601	-3.098	90	.003	9.597	.003	0.375
Liberals	threat	95	0.381	0.714	0.534	90	.595	0.285	.595	0.241
Conservatives	trust	86	-0.113	0.256	-0.441	81	.661	0.194	.661	0.416
Conservatives	threat	86	-0.159	0.311	-0.509	81	.612	0.260	.612	0.153

```
kable(stack_rows(lapply(mods, `[[`, "slopes")), digits = 3,
      caption = "Simple slopes of expected credit by empathy level")
```

Table 4: Simple slopes of expected credit by empathy level

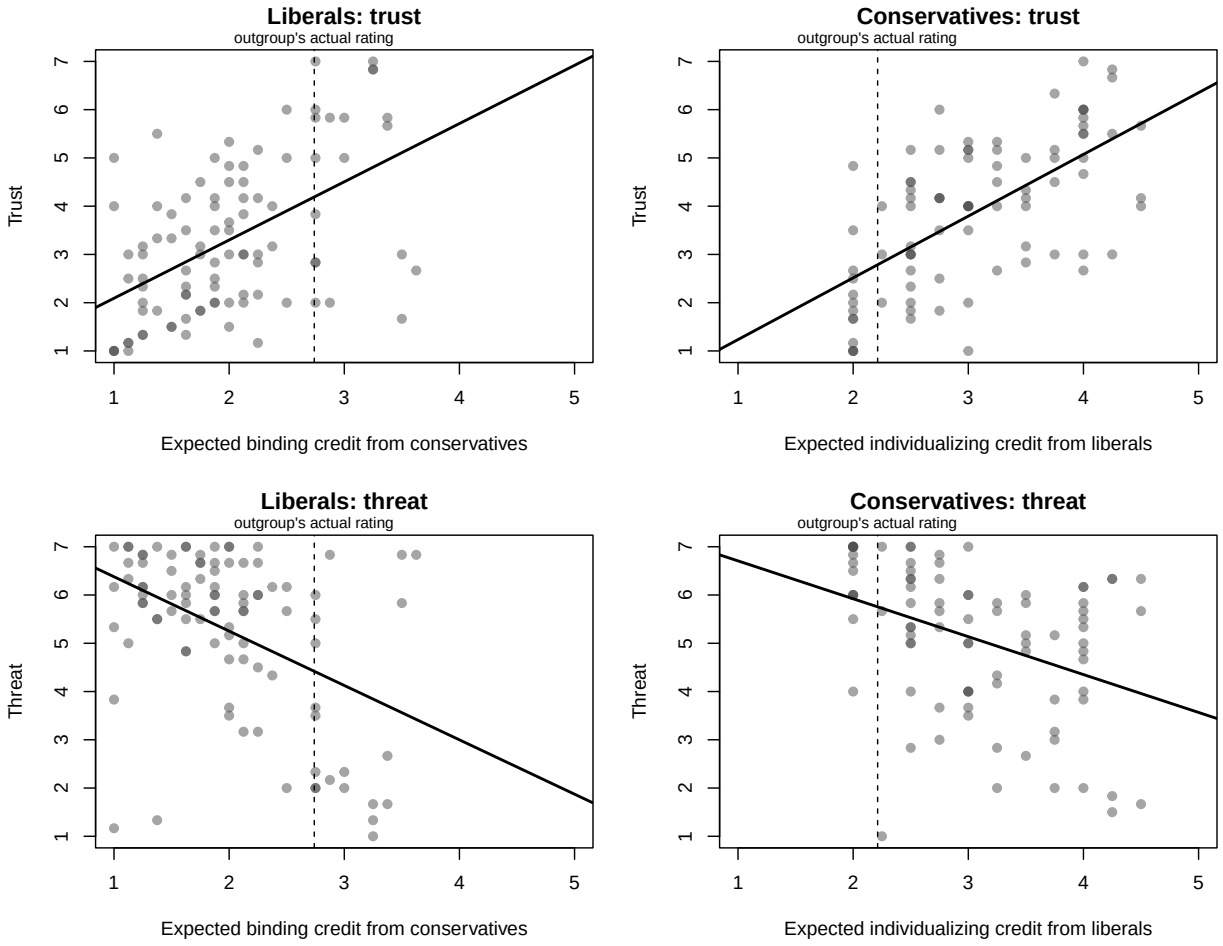
group	outcome	empathy_level	b	SE	t	p
Liberals	trust	-1 SD	1.596	0.269	5.924	< .001
Liberals	trust	Mean	0.978	0.217	4.519	< .001
Liberals	trust	+1 SD	0.361	0.317	1.137	.258
Liberals	threat	-1 SD	-1.081	0.320	-3.377	.001
Liberals	threat	Mean	-0.955	0.257	-3.710	< .001
Liberals	threat	+1 SD	-0.828	0.377	-2.197	.031
Conservatives	trust	-1 SD	1.313	0.238	5.522	< .001
Conservatives	trust	Mean	1.241	0.175	7.108	< .001
Conservatives	trust	+1 SD	1.170	0.239	4.901	< .001
Conservatives	threat	-1 SD	-0.684	0.289	-2.368	.020
Conservatives	threat	Mean	-0.784	0.212	-3.697	< .001
Conservatives	threat	+1 SD	-0.885	0.290	-3.050	.003

Figures

The dashed vertical line marks the outgroup's actual mean rating, i.e., where an accurate metaperception would sit. Regression lines are bivariate, for display only.

```
panel <- function(df, y, R, xlab, ylab, main) {
  plot(df$credit, df[[y]], xlab = xlab, ylab = ylab, main = main,
       pch = 19, col = adjustcolor("grey30", 0.5), xlim = c(1, 5),
       ylim = c(1, 7))
  abline(lm(df[[y]] ~ df$credit), lwd = 2)
  abline(v = R, lty = 2)
  mtext("outgroup's actual rating", side = 3, at = R, line = 0.1, cex = 0.7)
}

op <- par(mfrow = c(2, 2), mar = c(4.5, 4.5, 3, 1))
panel(lib_meta, "trust",  R_lib,
      "Expected binding credit from conservatives", "Trust", "Liberals: trust")
panel(con_meta, "trust",  R_con,
      "Expected individualizing credit from liberals", "Trust", "Conservatives: trust")
panel(lib_meta, "threat", R_lib,
      "Expected binding credit from conservatives", "Threat", "Liberals: threat")
panel(con_meta, "threat", R_con,
      "Expected individualizing credit from liberals", "Threat", "Conservatives: threat")
```



```
par(op)
```

Comparison with the values currently in the manuscript

```
run_version <- function(which) {
  s <- item_sets(which)
  d <- data
  d$binding         <- rowMeans(d[, s$binding], na.rm = composite_na_rm)
  d$individualizing <- rowMeans(d[, s$indiv],   na.rm = composite_na_rm)
  lm_ <- subset(d, Group == lib_code & Condition == cond_meta)
  cm_ <- subset(d, Group == con_code & Condition == cond_meta)
  lm_$credit <- lm_$binding
  cm_$credit <- cm_$individualizing
  ms <- list(
    lm(trust  ~ credit + ideology, data = lm_),
    lm(threat ~ credit + ideology, data = lm_),
    lm(trust  ~ credit + ideology, data = cm_),
    lm(threat ~ credit + ideology, data = cm_)
  )
  out <- stack_rows(Map(coef_row, ms, "credit",
    c("Liberals: trust", "Liberals: threat",
      "Conservatives: trust", "Conservatives: threat")))
  out$proportionality_in <- which
  out$p <- fmt_p(out$p)
  out[, c("proportionality_in", "model", "b", "SE", "t", "p", "R2")]
}

kable(rbind(run_version("individualizing"), run_version("binding")),
      digits = 3)
```

proportionality_in	model	b	SE	t	p	R2
individualizing	Liberals: trust	0.986	0.203	4.858	< .001	0.298
individualizing	Liberals: threat	-0.996	0.224	-4.445	< .001	0.265
individualizing	Conservatives: trust	1.342	0.249	5.396	< .001	0.286
individualizing	Conservatives: threat	-0.674	0.286	-2.354	.021	0.069
binding	Liberals: trust	1.052	0.225	4.686	< .001	0.288
binding	Liberals: threat	-0.947	0.253	-3.749	< .001	0.225
binding	Conservatives: trust	1.252	0.171	7.318	< .001	0.414
binding	Conservatives: threat	-0.776	0.208	-3.733	< .001	0.150

Manuscript-ready numbers

```
fmt_b  <- function(x) sprintf("%.2f", x)
fmt_r2 <- function(x) sub("^0", "", sprintf("%.2f", x))
fmt_pl <- function(p) ifelse(p < .001, "p < .001",
                             paste0("p = ", sub("^0", "", sprintf("%.3f", p))))

for (i in seq_len(nrow(new_tab))) {
  r <- new_tab[i, ]
  cat(sprintf("- **%s:** `($b = %s$, $%s$, $R^2 = %s$)`   SE = %.2f, t(%d) = %.2f\n",
              r$model, fmt_b(r$b), fmt_pl(r$p), fmt_r2(r$R2),
              r$SE, r$df, r$t))
}
```

- **Liberals: trust:** `($b = 1.05$, $p < .001$, $R^2 = .29$)` SE = 0.22, t(92) = 4.69
- **Liberals: threat:** `($b = -0.95$, $p < .001$, $R^2 = .23$)` SE = 0.25, t(92) = -3.75
- **Conservatives: trust:** `($b = 1.25$, $p < .001$, $R^2 = .41$)` SE = 0.17, t(83) = 7.32
- **Conservatives: threat:** `($b = -0.78$, $p < .001$, $R^2 = .15$)` SE = 0.21, t(83) = -3.73

Session info

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24)
Platform: aarch64-apple-darwin23
Running under: macOS Tahoe 26.5.2

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib;  LAPACK version 3.12.1

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Los_Angeles
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
[1] knitr_1.51       effectsize_1.0.2 broom_1.0.13     readxl_1.5.0.1

loaded via a namespace (and not attached):
 [1] jsonlite_2.0.0     dplyr_1.2.1        compiler_4.6.0     tidyselect_1.2.1
 [5] tidyr_1.3.2        yaml_2.3.12        fastmap_1.2.0      R6_2.6.1
 [9] generics_0.1.4     backports_1.5.1    tibble_3.3.1       insight_1.5.2
[13] pillar_1.11.1      rlang_1.2.0        xfun_0.59          parameters_0.29.2
[17] datawizard_1.3.1   otel_0.2.0         estimability_2.0.0 cli_3.6.6
[21] magrittr_2.0.5     emmeans_2.0.3      digest_0.6.39      mvtnorm_1.4-1
[25] rstudioapi_0.19.0  lifecycle_1.0.5    vctrs_0.7.3        evaluate_1.0.5
[29] glue_1.8.1         cellranger_1.1.0   bayestestR_0.18.1  rmarkdown_2.31
[33] purrr_1.2.2        tools_4.6.0        pkgconfig_2.0.3    htmltools_0.5.9
```